

DocuSense AI: Document Summary Assistant

Technical Assessment Approach Write-Up

Advait Varhade

<https://github.com/AdvaitVarhade/docusense-ai>

August 2026

Executive Approach Summary (≤ 200 Words)

DocuSense AI was engineered using a **Hexagonal (Ports & Adapters) Architecture** on **Next.js 15 App Router**, **TypeScript**, **Tailwind CSS**, and **shadcn/ui**, establishing strict boundaries between domain models, application use cases, and external infrastructure adapters.

To process heterogeneous document formats cost-effectively, I designed a **Tri-Engine Adaptive Extraction Pipeline**:

1. **Digital PDFs**: Parsed in-memory via WebAssembly (**unpdf**) in $< 50ms$ with zero AI token consumption.
2. **Scanned Images & Receipts**: Extracted client/server-side via **tesseract.js** for zero-cost offline OCR.
3. **Complex Layouts**: Automatically routed to Multimodal Vision AI (**Gemini Flash Vision**) for dense tabular data and handwriting.

For summarization, the system employs **Server-Sent Events (SSE)** to stream tokens in real-time ($TTF\!T < 450ms$), supporting **Short** (~ 150 words), **Medium** (~ 400 words), and **Long** (~ 900 words) presets with highlighted key takeaways and actionable improvement critiques.

To guarantee zero downtime and 100% free-tier sustainability, I implemented an inside-stream **multi-model fallback chain** (**gemini-3.6-flash** $\rightarrow$ **gemini-3.5-flash-lite** $\rightarrow$ **mock.offline_engine**), XML prompt-injection shields, comprehensive error boundaries, and a client-side multi-format export suite (Markdown, JSON, Plain Text, Clipboard).